

Automated Attack Graph Construction for Cross-host Threat Detection Using Cyber Threat Intelligence

Chen Ling^{†‡}, Yifei Li^{†‡}, Ziqing Feng^{†‡}, Bin Liu^{†‡}, Qiuyun Wang^{†‡},
Liling Xin^{†*}, Zhengwei Jiang^{†‡}, Huamin Feng[§]

[†]Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

[‡]School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

[§]Beijing Electronic Science and Technology Institute, Beijing, China

{lingchen, liyifei, fengziqing, liubin, wangqiuyun, xinliling, jiangzhengwei}@iie.ac.cn, fenghm@besti.edu.cn

Abstract—Cyber Threat Intelligence (CTI) reports provide valuable insights into cyber threats. However, manually constructing attack graphs from the unstructured CTI reports requires significant human effort. With the development of Large Language Models (LLMs), researchers have begun to harness LLMs for attack graph construction from CTI reports. Nevertheless, existing works mainly focus on describing abstract and high-level attack behaviors through these graphs, which cannot be used as query graphs for threat detection based on graph matching, as provenance graphs are at the system level. Moreover, these works do not consider modeling cross-host attack behaviors. To address these problems, we propose a novel method for automatically constructing attack graphs from CTI reports. We utilize prompt engineering, and leverage the in-context learning ability of LLMs to generate attack graphs. In this method, we first restructure the CTI report by grouping continuous sentences with the same tactics, and then use multiple LLMs to extract entities and relations. Finally, we use an LLM to integrate all the results. We also design a cross-host threat detection algorithm using the generated attack graphs. The evaluation results show that our method constructs attack graphs with an average conversion rate of 72.4%. It also achieves nearly 94.8% precision and 96.5% recall for IoC and relation extraction compared with manually labeled results.

Index Terms—cyber threat intelligence analysis, attack graph construction, large language model, in-context learning, prompt engineering

I. INTRODUCTION

In recent years, the increasing number of advanced cyber attacks has been raising concerns among security researchers. These sophisticated cyber attacks, such as Advanced Persistent Threats (APTs), typically involve multiple stages, among which cross-host attack behaviors—i.e., lateral movements—are particularly critical [1]. To better combat these attacks, security analysts widely utilize Cyber Threat Intelligence (CTI), which contains informative knowledge, including attack patterns, and Indicators of Compromise (IoCs) [2].

This work is partially supported by Youth Innovation Promotion Association, CAS (No. 2023170) and Beijing Key Laboratory of Network Security and Protection Technology.

*Corresponding author.

Traditional CTI is generally defined by open standards (e.g., STIX [3]), and represented as a limited set of IoCs, including IP addresses, domains, malicious file names, and hashes. Nevertheless, such CTI fails to capture the interrelationships between intelligence entities. Instead, the CTI reports written in natural language provide more detailed accounts of real-world attacks, making them especially valuable for researchers [4]. The attack graph, as a structured representation of information extracted from CTI reports, has gained increasing attention. It models the cyber attack process through threat entities and their relations, enabling more robust [5] and in-depth graph-based threat detection. However, extracting information from the large volume of CTI reports requires significant manual efforts, because these reports commonly come in non-uniform and unstructured formats. To overcome this challenge, considerable efforts have been devoted to designing automated extraction methods using deep learning and Natural Language Processing (NLP) techniques [6]–[10], but these methods require specialized design with a large volume of labeled data for training a model, thereby showing limited generalization capabilities when facing emerging attack scenarios.

The advent of Large Language Models (LLMs) provides a more promising solution. LLMs are pre-trained on massive datasets, endowing them with strong contextual understanding and knowledge reasoning capabilities. Moreover, LLMs can perform various zero-shot tasks through in-context learning. Therefore, recent studies have investigated using LLMs to automate the construction of attack graphs [4], [11], [12] based on CTI reports. However, these works mainly focus on building attack graphs that describe abstract, high-level attack behaviors. In contrast, many threat hunting methods rely on provenance graphs built from low-level system audit logs [13], [14]. Due to this semantic gap, the generated attack graphs cannot be applied as query graphs [15] for matching against provenance graphs. As a result, the construction of query graphs still demand substantial human efforts.

Another limitation of current works is that, most graph-based threat hunting methods focus on a single host, resulting in the exclusion of cross-host attack behaviors [13]–[16].

Moreover, these methods typically extract subgraphs starting from nodes that match any IoCs, and then compare the extracted subgraphs with given query graphs. As a result, current studies cannot detect cross-host threats without IoCs related to lateral movement behaviors. Therefore, how to model cross-host threats in the query graph remains an open question.

In this paper, we propose a novel solution to automatically construct attack graphs from CTI reports for cross-host threat detection. It takes a CTI report as input, and generates an attack graph that can be used as a query graph for graph matching tasks. Furthermore, we introduce a new entity in the attack graph to represent lateral movement behaviors. The attack graph construction process consists of three phases, including report restructuring, information extraction, and results integration. It makes use of the powerful in-context learning of LLMs. The report restructuring rewrites the unstructured CTI report to filter out noisy information. Next, the attack entities and relations are extracted, and the results are returned in a structured format suitable for building an attack graph. Then we integrate the results generated by different LLMs to produce the final graph. Additionally, we design a cross-host threat detection algorithm using the generated attack graphs.

The main contributions of this paper are as follows:

- We propose a novel LLM-based approach that automatically analyzes CTI reports and constructs attack graphs, which can be used for cross-host threat detection.
- We create a special entity to represent lateral movement behaviors, which facilitates the identification. Besides, we design a cross-host threat detection method based on graph similarity calculation, and use the generated attack graphs to perform the detection.
- We evaluate the effectiveness of automated graph construction on the AnnoCTR dataset [17]. The results show that our method achieves 72.4% conversion rate on average, with a precise extraction of IoCs and relations.

The remainder of this paper is organized as follows. In section II, we introduce the related work on CTI report extraction and threat hunting approaches. In section III, we present our method for automatically constructing attack graphs, and illustrate the design of cross-host threat detection using the generated graphs. Section IV describes the experiment setup and evaluation results. Finally, section V concludes the paper.

II. RELATED WORK

A. CTI Report Extraction

Previous studies have focused on automatically extracting valuable information from CTI reports, and generating structured outputs via NLP techniques. iACE generates formatted IoC files [18], TTPDrill analyzes each threat action, and maps the action to proper techniques, tactics and kill chain phases [19], while Nguyen *et al.* utilize a SecBERT-based model for mapping CTI reports to specific ATT&CK techniques [20], [21]. In addition, recent studies have explored transforming CTI reports into graph-based representations. Extractor and ThreatRaptor use NLP techniques to analyze attack behaviors

and construct attack graphs [6], [22]. Zhou *et al.* use pre-trained language model T5 [23] to design a multi-hop reasoning framework for building knowledge graphs [24]. AttackKG applies a learning-based Named Entity Recognition (NER) model and a natural language parsing model to extract entity and relation information, and constructs technique knowledge graphs [7]. However, these NLP techniques show limited capabilities in semantic understanding, and often require a large amount of labeled data to train the model.

With the development of LLMs, researchers have increasingly investigated methods for CTI report extraction by leveraging the in-context learning of LLMs. In-context learning refers to the ability to learn from a few examples in the context, without the need for additional training phases [25]. LLMCloudHunter proposes a framework for generating sigma rules from CTI reports [26]. CTIKG divides CTI reports into multiple text segments, and then employs LLM agents to perform triple extraction from the text segments and summarize the results with a dual memory mechanism [4]. LLM-TIKG uses LLMs to extract entities and relations, and performs TTP classification to build knowledge graphs [11]. AttackKG+ analyzes CTI reports to build attack graphs, and introduces a new attack knowledge schema that represents a cyber attack as a temporally unfolding event [12]. However, these graphs built by the above methods represent knowledge about high-level attack behaviors, thus cannot be used for graph matching process of threat detection.

B. Threat Hunting Approaches

Existing threat hunting approaches aim to detect attack traces through sequence extraction and classification [27], rule-based matching [28], exact graph matching [22], or graph similarity matching [13]–[15]. Bhattarai and Huang illustrate that the graph better demonstrates attack activities than the sequence [28], and many threat detection systems [13], [15], [28]–[30] suggest that detecting malicious subgraphs instead of the whole provenance graph generates more explainable alarms for analysts. Moreover, Aly *et al.* indicate that, graph similarity matching method, as a promising solution, balances the trade-off between the needs of domain experts and detection precision [14]. These threat hunting methods based on graph similarity matching require query graphs, including IoCs and their relations. Nevertheless, the query graphs should be aligned with the provenance graph in terms of the node and edge types, so they are different from those knowledge graphs generated from CTI reports, which describe abstract, high-level attack behaviors.

For cross-host threat detection, King *et al.* propose a basic approach by identifying network connections, and then adding inter-host causal dependencies between processes that handle the connections [31]. Ji *et al.* design a taint analysis method based on tag propagation to detect cross-host attacks [32]. SteinerLog uses coarse-grained network-wide graphs to track the movement of attackers [28]. However, these methods do not leverage the effective information from textual CTI reports. Moreover, the taint analysis methods incur significant

overhead [5], and SteinerLog only tracks compromised users and hosts rather than specific cross-host behaviors. Hence, in this paper, we focus on automatically generating attack graphs from CTI reports, and applying graph similarity matching methods for cross-host threat detection. These attack graphs serve as query graphs, which are then matched against the suspicious subgraphs extracted from the provenance graph.

III. METHODOLOGY

In this section, we first present the ontology model of the attack graph. Secondly, we illustrate our solution for constructing attack graphs via LLMs. As shown in Fig. 1, we divide our approach into three phases, i.e., CTI report restructuring, information extraction, and results integration. We leverage the in-context learning capability of LLMs by providing extraction rules and examples in the instructions. Thirdly, we propose a cross-host threat detection method which is based on graph similarity calculation using the generated attack graphs. The full prompts are available at <https://github.com/hmdnhwanLct/auto-ag-construction>.

A. Ontology Design

The ontology model is an explicit specification of the attack graph [33]. It defines the categories of entities and their relations included in the attack graph. To ensure consistency with provenance graphs derived from audit logs, the categories of entities and relations should be defined at the system-level granularity. We focus on concrete objects such as processes and files, instead of abstract entities such as attack techniques and organizations.

Therefore, as shown in Table I, we define five categories of entities: process, file, socket, registry, and a special category, LM, which represents lateral movement behaviors. In addition, each type of entity has multiple attributes. We also define 21 categories of relations shown in Table II. These relations describe interactions between two entities. Note that PROC_CON_HOST, FILE_SND_HOST, FILE_RCV_HOST, and SOCK_CON_HOST are related to lateral movement behaviors. In the attack graph, each entity is represented as a node, and each relation is represented as an edge.

TABLE I
ENTITY CATEGORIES AND CORRESPONDING ATTRIBUTES

Entity Type	Attributes
Process	name, path, arguments, cmdline, PID, parentPID, startTime, ownerUser
File	name, path, permissions, size, MD5, SHA1, SHA256, ownerUser, ownerGroup, signature, creationTime
Socket	localIP, localPort, remoteIP, remotePort, associatedProcess, domainName, protocol
Registry	key, value
LM	technique

TABLE II
RELATION CATEGORIES

Relation Type	Description
PROC_CRE_FILE	Process creates file
PROC_LOAD_FILE	Process loads file
PROC_DEL_FILE	Process deletes file
PROC_WRT_FILE	Process writes to file
PROC_RD_FILE	Process reads file
FILE_EXE_PROC	File is executed as process
PROC_BND_PORT	Process binds to port
PROC_CRE_CONN	Process initiates connection
PROC_SR SOCK	Process sends/receives data via socket
PROC_CLS SOCK	Process closes socket
PROC_SPN_PROC	Process spawns child process
PROC_INJ_PROC	Process injects into another process
PROC_IMP_PROC	Process impersonates another process
PROC_COM_IPC	Process communicates via IPC
PROC_TRM_PROC	Process terminates another process
PROC_DUP_HNDL	Process duplicates handle to another process
PROC_EDT_REG	Process edits registry
PROC_CON_HOST	Process connects to other hosts
FILE_SND_HOST	File is sent to other hosts
FILE_RCV_HOST	File is received from other hosts
SOCK_CON_HOST	Socket connects to other hosts

B. Phase 1: Report Restructuring

In order to filter out noisy information from the original CTI report, we start by restructuring the report instead of directly extracting entities and relations. We only keep sentences that contain the technical details of the attack in temporal order. These sentences are then labeled by attack tactics defined by MITRE [21]. Next, we group continuous sentences with the same label together.

To achieve this, the prompt we design encompasses the following two parts:

- **Restructuring rules.** We instruct the LLM to rewrite the report, and specify the output format with examples. Moreover, we provide the ontology model, and ask the LLM to retain sentences that contain key information about these entities and relations.
- **Definitions of tactics.** For each tactic, we provide a corresponding definition and description for the LLM, so that it can use these tactics to label a sentence. Note that the first two tactics, i.e., Reconnaissance and Resource Development, are excluded for the following reasons. First, we observe that they rarely appear in CTI reports. Second, both tactics serve as preparatory stages for subsequent attacks: Reconnaissance focuses on gathering information, and Resource Development involves organizing resources. Since we mainly concentrate on automatically generating attack graphs for threat hunting based on host log analysis, these tactics are beyond the scope of this study.

C. Phase 2: Information Extraction

The information extraction phase aims to extract entities and relations from the restructured reports. The output consists of a list of entities with attributes, and a list of relations. Each element of the relation list is presented as a triple $\langle \text{subject}, \text{predicate}, \text{object} \rangle$, where subject

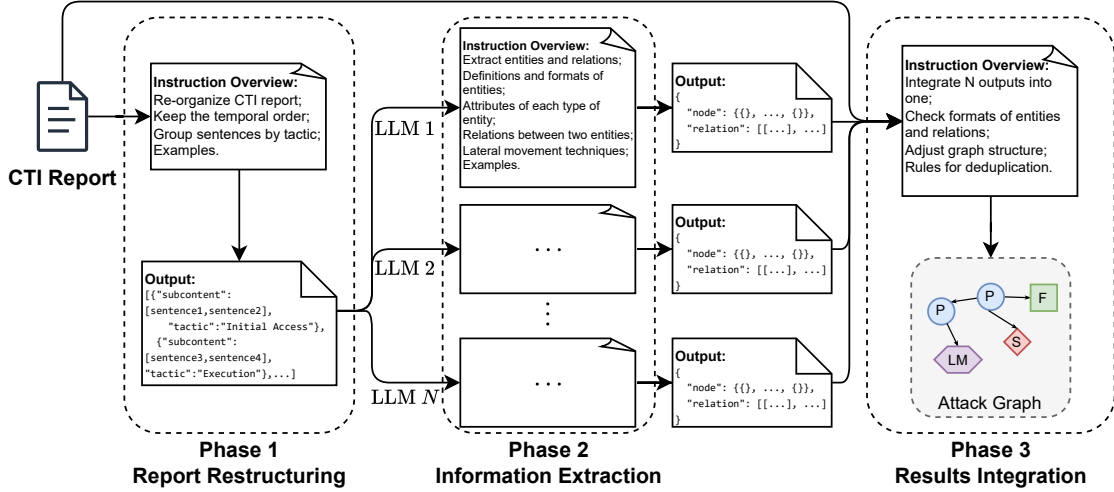


Fig. 1. The overall framework of automated attack graph construction.

and `object` are entities, and `predicate` is a relation. We observe that some CTI reports do not always follow the temporal order, and the same entity may appear multiple times. For example, a report may provide details about a malicious binary file in one section, and describe attack behaviors, such as how the file is created, in a subsequent section. This can lead LLMs to mistakenly create duplicate entities or overlook some entities. To address this challenge, we instruct LLMs to ensure that the output forms an undirected attack graph, because the undirected graph can weaken the temporal causality between relations. Since the relations defined in the ontology model inherently encode subject-object roles, the undirected graph can be easily converted to a directed one. Additionally, we employ multiple LLMs to perform information extraction. We assume that the number of LLMs used in this phase is N .

The prompt is constructed to cover the following aspects:

- **Entity extraction rules.** We instruct the LLMs to extract entities based on the given descriptions of each type of entity. We also specify the attributes of each entity defined in the ontology model. For lateral movement entities (LM), we provide nine types of lateral movement techniques defined by MITRE along with related descriptions. Moreover, we ask LLMs to replace the pronouns such as “it”, “they”, “attacker”, with specific entities as much as possible according to the context.
- **Relation extraction rules.** We restrict all relation types to those defined in the ontology model. Another rule is that an LM entity can only have a single edge connected to a process, file, or socket entity. This rule aims to explicitly specify the entity that initiates a lateral movement.

D. Phase 3: Results Integration

To mitigate the hallucinations and randomness of LLMs, we use an LLM to integrate all the results generated in the information extraction phase, producing a comprehensive and

structured result for attack graph construction. The input data consist of N results, and the original CTI report.

The prompt can be divided into two parts:

- **Examination Rules.** We repeat the output format, the categories of entities and relations, and instruct the LLM to re-check and correct mistakes based on the original report. To correct the inconsistent relations, such as those between two files or a file and a socket, we ask LLMs to add a process entity with the same name as the initiator.
- **Merge Rules.** After examination, the LLM should identify entities from N different results that represent the same entity, and merge them into one. The merged entity inherits the union of attributes from the original entities. Additionally, if multiple entities of the same type are connected to the same entity, and the types of their attributes are the same while only the values are different, these entities should be merged into one. Then we remove duplicate relations. Finally, we ask the LLM to remove all the isolated nodes. If the depth of a generated graph is smaller than 2, we discard the graph.

E. Cross-Host Threat Detection Using Attack Graphs

We briefly outline our idea of using attack graphs for cross-host threat detection. Our method is based on graph similarity calculation, as shown in Algorithm 1. We begin by extracting IoCs from an attack graph, i.e., the values of attributes of all entities at line 4.

If an IoC is matched in the provenance graph PG , at lines 6–8, we extract a suspicious subgraph starting from the suspicious node related to the IoC, and save the corresponding node and the subgraph to lists respectively. We first traverse multiple hops h from the suspicious node. During the traversal process, we only retain process nodes, socket nodes, and IoC-related nodes, as well as their corresponding edges.

At line 9, we prune the original attack graph according to the matched suspicious nodes. The motivation is that many IoCs

Algorithm 1: Cross-host Threat Detection Algorithm

Input: attack graph AG , provenance graph PG , similarity threshold θ

Output: a list of detected malicious subgraphs MG

```

1  $SN = []$  // list of suspicious nodes
2  $SG = []$  // list of suspicious subgraphs
3  $QG = []$  // list of query graphs
4 foreach node  $n \in AG$ , attribute value  $a \in n$  do
5   if  $a$  is not matched in  $PG$  then continue
6    $SN.append((n, a))$ 
   // extract a subgraph from an IoC
7    $sg = \text{ExtractSuspiciousSubgraph}(a, PG)$ 
8    $SG.append(sg)$ 
9  $PrunedGraphs = \text{AGPruning}(AG, SG, SN)$ 
10  $QG = \text{RemoveLMEntity}(PrunedGraphs)$ 
11 for  $i = 0; i < SG.length; i++$  do
12    $sg = SG[i], qg = QG[i], sn = SN[i]$ 
13    $s = \text{ComputeGraphSimilarity}(sg, qg)$ 
14   if  $s > \theta$  then // if exceeds threshold
15      $\text{LMAAnalysis}(sg, PrunedGraphs[i], sn)$ 
16      $MG.append(sg)$ 

```

are listed in the CTI report, but only some of them occur in practice. As a result, the similarity scores may be reduced even when the suspicious subgraphs and attack graphs are highly similar. To address this problem, for each suspicious node, we check its sibling nodes. If a sibling node is not related to any IoC, and has the same number of attributes and the same types of attributes, we prune the node from the attack graph. Since there is no lateral movement node in the provenance graph, we remove LM nodes from the attack graph at line 10. By pruning these nodes and their corresponding edges, we obtain the corresponding query graph for each suspicious subgraph.

At lines 11–16, we compute the graph similarity score using a suspicious subgraph and its corresponding query graph. Following the graph representation learning component introduced in [14], we use only the types of nodes and edges as features, then we apply RGCN [34] for graph embedding and Neural Tensor Network (NTN) [35] to compute the similarity score. During the training phase, we train the model on randomly extracted benign subgraphs, and use Graph Edit Distance (GED) as the similarity metric, thus the model is robust against imbalanced datasets. If the score exceeds the predefined threshold θ , we mark the suspicious subgraph as a malicious subgraph. Next, we perform lateral movement analysis for cross-host threat detection at line 15. If the attack graph contains a path of length d from the IoC-related node to an LM node, and the suspicious graph sg contains a path of the same length from the IoC-related node to a socket node, which represents a connection to an intranet IP, we assign the technique attribute of the LM node to this connection. If audit logs are also collected from the device associated with the intranet IP, we can find a socket node corresponding to

the one in sg , and extract a subgraph to further investigate the attack behaviors.

IV. EVALUATION

In this section, we provide a detailed evaluation of our automated attack graph construction method. We use the AnnoCTR dataset [17] to evaluate the effectiveness of our method. Specifically, we measure the conversion rate from CTI reports to attack graphs, as well as the evaluation metrics used for assessing the accuracy of IoC and relation extraction. We also present a case study for cross-host threat detection.

A. Evaluation Setup

We use the AnnoCTR dataset [17] for evaluation. AnnoCTR dataset contains 400 CTI reports in multiple formats collected from Intel471, Zscaler and other commercial CTI vendors. We use the plain text version as the input of LLMs. In our experiments, we set the number of LLMs $N = 3$, and use DeepSeek-V3-0324, Doubao-seed-1.6-thinking, and Gemini-2.5-flash to process CTI reports and construct attack graphs. Note that only Doubao-seed-1.6-thinking applies the Chain-of-Thought (CoT). We use Gemini-2.5-flash for report restructuring, and use all of these LLMs for information extraction. Then, we apply each LLM for results integration. We also note that all the CTI reports analyzed in this paper are publicly available. For private security reports and other resources, self-deployed LLMs or official LLM APIs are highly recommended to avoid potential data leakage.

To evaluate the performance of our method, we measure the conversion rate from CTI reports to attack graphs. We further evaluate its capability to reduce noisy information by counting the number of extracted IoCs. Additionally, we manually analyze 15 CTI reports to compute precision, recall, and F1 score for the efficacy of IoC and relation extraction.

- **Precision**, which is defined as:

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (1)$$

- **Recall**, which is defined as:

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (2)$$

- **F1 score**, which is defined as:

$$\text{F1 score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (3)$$

Where TP is the number of IoCs and relations that are extracted by our method, FP is the number of extracted IoCs and relations that do not exist in the manual analysis result, and FN is the number of IoCs and relations that are not extracted by our method.

TABLE III
CONVERSION RATES FROM CTI REPORTS TO ATTACK GRAPHS

Model Used in Phase 3	Valid Results#	Conversion Rate
Gemini-2.5-flash	307	0.768
DeepSeek-V3-0324	303	0.758
Doubao-seed-1.6-thinking	258	0.645

TABLE IV
EVALUATION METRICS OF IOC AND RELATION EXTRACTION

CTI Reports	Manual Analysis		Our Method			Metrics		
	IoCs	Relations	IoCs	Relations	Duplicate IoCs	Precision	Recall	F1 Score
lab52_2019-05-16	16	15	23	15 (+1, -1)	7	0.968	0.968	0.968
lab52_2020-03-23	6	12	2 (-4)	15 (+3)	0	0.824	0.778	0.800
lab52_2020-08-26	15	16	31	15 (-1)	16	1	0.968	0.984
proofpoint_2020-05-20	3	24	9 (+3)	25 (+1)	3	0.871	1	0.931
proofpoint_2020-10-06	1	11	2	12 (+1)	1	0.923	1	0.960
proofpoint_2020-10-21	8	9	9 (+1, -1)	10 (+1)	1	0.889	0.941	0.914
proofpoint_2021-01-26	12	25	12	24 (-1)	0	1	0.973	0.986
proofpoint_2021-06-17	7	37	6 (+1, -2)	38 (+1)	0	0.955	0.955	0.955
proofpoint_2022-01-27	3	16	7	16 (+1, -1)	4	0.947	0.947	0.947
quoteintelligence_2020-08-07	14	14	14	14	0	1	1	1
zscaler_2013-07-11	12	13	11 (-1)	13	0	1	0.960	0.980
zscaler_2013-09-18	20	26	22	26	2	1	1	1
zscaler_2014-09-29	25	23	30	23	5	0.979	1	0.989
zscaler_2019-11-21	4	14	4	15 (+1)	0	0.947	1	0.973
zscaler_2021-03-30	24	37	26 (+2)	39 (+3, -1)	0	0.923	0.984	0.952

B. Effectiveness of Graph Construction

1) *Conversion Rates*: We present the conversion rates achieved by different models in the results integration phase. After the report restructuring and information extraction phase, there are 279 valid results generated by Gemini, 261 by DeepSeek, and 281 by Doubao. After the results integration phase, as shown in Table III, the number of valid results changes to 307, 303, and 258 for Gemini, DeepSeek, and Doubao, respectively. LLMs without CoT prompting achieve conversion rates exceeding 75%, whereas Doubao, which employs CoT prompting, achieves a conversion rate of 64.5%. The results indicate that LLMs without CoT prompting are sufficient for the graph construction task, achieving higher conversion rates, while CoT may induce unnecessary reasoning that hinders conversion performance. Additionally, during the results integration phase, the LLM benefit from multiple extraction outputs, leading to a higher number of valid results for Gemini and DeepSeek compared to the outputs of the information extraction phase. We also analyze the reports that fail to convert into attack graphs, and find that most of these reports contain only security trends or security suggestions, without any details of IoCs. For example, suggestions to avoid phishing emails, or results of a survey.

TABLE V
NUMBER OF IDENTIFIED IOCS AT EACH PHASE

Phase	Model	Identified IoCs
N/A (Original Reports)	N/A	13457
Report Restructuring	Gemini-2.5-flash	6534
	Gemini-2.5-flash	4803
Information Extraction	DeepSeek-V3-0324	4143
	Doubao-seed-1.6-thinking	4072
	Gemini-2.5-flash	5167
Results Integration	DeepSeek-V3-0324	4765
	Doubao-seed-1.6-thinking	4866

2) *Noisy Information Reduction*: To demonstrate the ability to remove noisy information, we use a command line tool, iocsearcher [36], which is an IoC extractor based on regular expression matching, to extract all potential IoCs from

CTI reports. For the original CTI reports and the outputs at each phase, we evaluate the number of IoCs identified by iocsearcher. As shown in Table V, iocsearcher detects an average of 4339 IoCs from the results of the information extraction phase. After the results integration phase, the number of identified IoCs increases to an average of 4933, reducing approximately 63.3% of the invalid information from the original reports. This is because the LLM, i.e., Gemini-2.5-flash in this experiment, integrates the three results and produces a comprehensive output.

3) *Accuracy of IoC and Relation Extraction*: Moreover, we manually analyze 15 CTI reports from AnnoCTR Dataset to evaluate the Accuracy of IoC and relation extraction. As shown in Table IV, the “Manual Analysis” column indicates the manually labeled number of IoCs and relations, and “Our Method” column presents the number of IoCs and relations identified in the generated attack graphs. In this experiment, we use Gemini-2.5-flash during the results integration phase. We mark the number of false positives in green, with a plus sign preceding the number (e.g., +1), and mark the number of false negatives in red, with a minus sign preceding the number (e.g., -1). We compute the precision, recall, and F1 score to quantify the accuracy of the information extraction. Note that we compute these metrics for both IoCs and relations, but we do not take the duplicate IoCs into consideration.

The results in Table IV show that our method achieves an average of 94.8% precision and 96.5% recall, indicating a high accuracy in extracting IoCs and associated relations from unstructured CTI reports. In particular, our method demonstrates robust and consistent performance across these reports, including those from different vendors. For several reports, such as quoteintelligence_2020-08-07 and zscaler_2013-09-18, our method achieves an F1 score of 1. We also observe that our method generates many duplicate IoCs in some CTI reports, with up to 16 duplicates in one case. These duplicates are primarily attributed to the inclusion of multiple URLs that point to the same IP, or different formats of file paths that represent the same file.

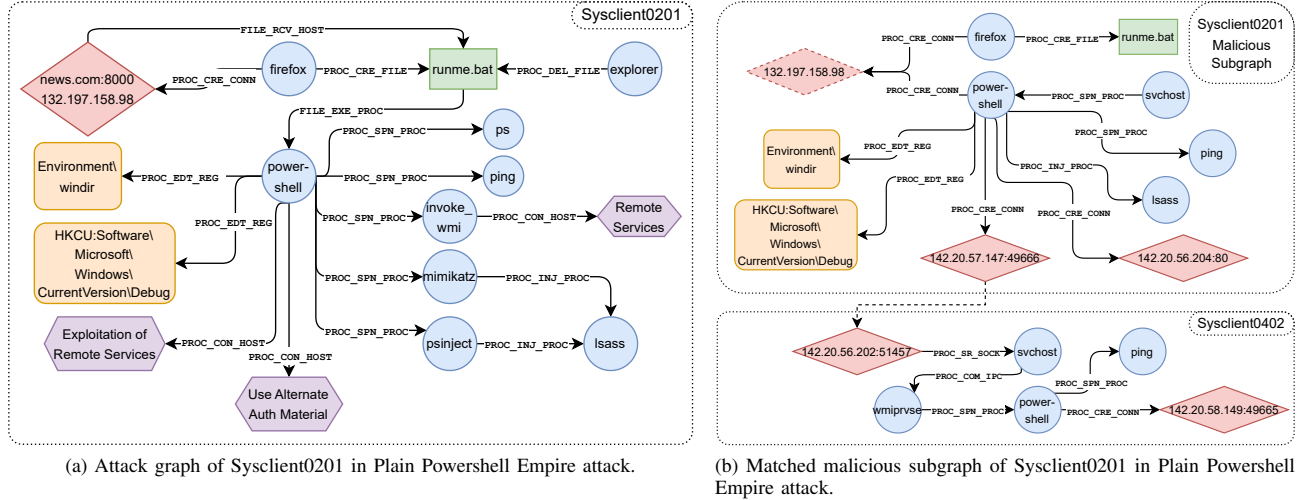


Fig. 2. Case study of Plain Powershell Empire attack in DARPA OpTC dataset. Some unrelated nodes and edges are omitted for simplicity.

4) *Processing Time of Graph Construction*: The processing time depends on the length of CTI reports and the type of model. For the report restructuring phase, Gemini-2.5-flash and DeepSeek-V3-0324 take an average of 9 seconds to generate the results, while Doubao-seed-1.6-thinking, which applies CoT prompting, is considerably slower, taking approximately 80 seconds. For the information extraction and results integration phases, Gemini-2.5-flash and DeepSeek-V3-0324 require 5 seconds on average, whereas Doubao-seed-1.6-thinking requires 40 seconds. Therefore, our method only uses the CoT-based model in the information extraction phase, and achieves high efficiency with the overall attack graph construction taking about 54 seconds on average.

C. Case Study of Cross-Host Threat Detection

We provide a case study that demonstrates the application of our constructed attack graph in a cross-host threat detection scenario. We utilize audit logs from the DARPA OpTC dataset [37]. This dataset contains more than 17 billion events, which were collected from one thousand hosts over a seven-day period. Each host ran on Windows 10, and was configured to emulate normal user activities. During the last three days, the red team conducted three simulated APT attacks. We selected the first attack, i.e., the “Plain Powershell Empire” attack, and applied our method to construct an attack graph on host Sysclient0201 based on the attack report. Then, we built a provenance graph based on the system audit logs from host Sysclient0201, and set the number of traversing hops $h = 2$ to extract suspicious subgraphs.

During the Plain Powershell Empire attack, a normal user accessed news.com:8000, and downloaded a malicious file named runme.bat via the Firefox browser, and then executed its malicious code, which created a powershell process for remote command execution. Attackers performed a registry modification of the “windir” value in “Environment” to bypass User Account Control (UAC), executing a series of commands to

collect credentials and system information, such as mimikatz, ping, psinject, etc. Attackers also attempted to connect to 142.20.58.149 via SMB. Finally, attackers used a powershell script named invoke_wmi to perform lateral movement, and logged into host Sysclient0402.

Fig. 2 visualizes a simplified attack graph and a matched malicious subgraph. We present processes, files, registry, socket, and LM nodes by circles, rectangles, rounded rectangles, diamonds, and hexagons, respectively. The dotted diamond node in Fig. 2b, which contains IP 132.197.158.98, is the starting point of the malicious subgraph. As shown in Fig. 2a, the constructed attack graph contains most of the attack behaviors, although some nodes and relations are mistakenly generated by LLMs. For example, the malicious subgraph shown in Fig. 2b does not include nodes such as ps, psinject, mimikatz, and invoke_wmi. We find that these are powershell scripts executed in the powershell process, thus they do not exist as process nodes in the malicious subgraph.

After detecting malicious subgraphs on a single host, we further analyze the cross-host behaviors. As described in section III-E, we correlate the socket 142.20.57.147:49666 (host Sysclient0402) and 142.20.56.204:80 with two LM entities, whose technique attributes are “exploitation of remote services” and “use alternate authentication material”. We then find the corresponding socket in Sysclient0402, and extract a subgraph that only contains processes and sockets shown in Fig. 2b. The subgraph indicates that attackers performed a ping sweep attack, and then moved to another host. The case study shows that the attack graph generated via our method effectively identifies IoCs and their relations, while it also covers the cross-host threat by leveraging LM entities.

V. CONCLUSION

In this paper, we propose an automated attack graph construction method for cross-host threat detection. We provide meticulously designed instructions with examples, and lever-

age the in-context learning of LLMs for restructuring the original CTI reports, extracting entities and relations, and integrating the results from multiple sources. In addition, we design a cross-host threat detection algorithm that makes use of the attack graphs, by calculating graph similarity scores between the generated attack graphs and the suspicious sub-graphs extracted from a system-level provenance graph. We evaluate our method based on real-world CTI reports. The results show that our graph construction method successfully converts CTI reports into attack graphs with an average conversion rate of 72.4%. Moreover, our method achieves a precision of 94.8% and a recall of 96.5% for IoC and relation extraction when compared with the manually labeled results.

REFERENCES

- [1] I. J. King and H. H. Huang, "Euler: Detecting network lateral movement via scalable temporal link prediction," *ACM Transactions on Privacy and Security (TOPS)*, vol. 26, no. 3, 2023.
- [2] R. Brown and K. Nickels. (2023) Sans 2023 cti survey: Keeping up with a changing threat landscape. [Online]. Available: <https://www.sans.org/white-papers/2023-cti-survey-keeping-up-changing-threat-landscape/>
- [3] OASIS Open. Introduction to STIX. [Online]. Available: <https://oasis-open.github.io/cti-documentation/stix/intro.html>
- [4] L. Huang and X. Xiao, "CTIKG: LLM-powered knowledge graph construction from cyber threat intelligence," in *First Conference on Language Modeling (COLM)*, 2024.
- [5] Z. Li, Q. A. Chen, R. Yang, Y. Chen, and W. Ruan, "Threat detection and investigation with system-level provenance graphs: A survey," *Computers & Security*, vol. 106, 2021.
- [6] K. Satvat, R. Gjomemo, and V. Venkatakrishnan, "Extractor: Extracting attack behavior from threat reports," in *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2021.
- [7] Z. Li, J. Zeng, Y. Chen, and Z. Liang, "AttacKG: Constructing technique knowledge graph from cyber threat intelligence reports," in *European Symposium on Research in Computer Security (ESORICS)*, 2022.
- [8] B. Cui, J. Li, and W. Hou, "ATDG: An automatic cyber threat intelligence extraction model of dpcnn and bigru combined with attention mechanism," in *International Conference on Web Information Systems Engineering (WISE)*, 2023.
- [9] K. Ahmed, S. K. Khurshid, and S. Hina, "CyberEntRel: Joint extraction of cyber entities and relations using deep learning," *Computers & Security*, vol. 136, 2024.
- [10] M. T. Alam, D. Bhusal, Y. Park, and N. Rastogi, "Looking beyond iocs: Automatically extracting attack patterns from external cti," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2023.
- [11] Y. Hu, F. Zou, J. Han, X. Sun, and Y. Wang, "LLM-TIKG: Threat intelligence knowledge graph construction utilizing large language model," *Computers & Security*, vol. 145, 2024.
- [12] Y. Zhang, T. Du, Y. Ma, X. Wang, Y. Xie, G. Yang, Y. Lu, and E.-C. Chang, "AttacKG+: Boosting attack graph construction with large language models," *Computers & Security*, vol. 150, 2025.
- [13] R. Wei, L. Cai, L. Zhao, A. Yu, and D. Meng, "DeepHunter: A graph neural network based approach for robust cyber threat hunting," in *Security and Privacy in Communication Networks (SecureComm)*, 2021.
- [14] A. Aly, S. Iqbal, A. Youssef, and E. Mansour, "MEGR-APT: A memory-efficient apt hunting system based on attack representation learning," *IEEE Transactions on Information Forensics and Security (TIFS)*, vol. 19, 2024.
- [15] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, "Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting," in *Proceedings of the 2019 ACM SIGSAC conference on Computer and Communications Security (CCS)*, 2019.
- [16] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "UNICORN: Runtime provenance-based detector for advanced persistent threats," in *27th Annual Network and Distributed System Security Symposium (NDSS)*, 2020.
- [17] L. Lange, M. Müller, G. H. Torbati, D. Milchevski, P. Grau, S. C. Pujari, and A. Friedrich, "AnnoCTR: A dataset for detecting and linking entities, tactics, and techniques in cyber threat reports," in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, 2024.
- [18] X. Liao, K. Yuan, X. Wang, Z. Li, L. Xing, and R. Beyah, "Acing the ioc game: Toward automatic discovery and analysis of open-source cyber threat intelligence," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.
- [19] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, and X. Niu, "TTPDrill: Automatic and accurate extraction of threat actions from unstructured text of cti sources," in *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC)*, 2017.
- [20] K.-D. Nguyen, H.-C. Chu, Q.-V. Nguyen, M.-T. Sun, K. Sakai, and W.-S. Ku, "From data to action: Cti analysis and att&ck technique correlation," in *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2024.
- [21] The MITRE Corporation. Mitre ATT&CK. [Online]. Available: <https://attack.mitre.org>
- [22] P. Gao, F. Shao, X. Liu, X. Xiao, Z. Qin, F. Xu, P. Mittal, S. R. Kulkarni, and D. Song, "Enabling efficient cyber threat hunting with cyber threat intelligence," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, 2021.
- [23] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research (JMLR)*, vol. 21, no. 140, 2020.
- [24] K. Zhou, Y. Xie, and X. Liu, "A multi-hop reasoning framework for cyber threat intelligence knowledge graph," in *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2024.
- [25] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li, H. Xia, J. Xu et al., "A survey on in-context learning," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [26] Y. Schwartz, L. Ben-Shimol, D. Mimran, Y. Elovici, and A. Shabtai, "LLMCloudHunter: Harnessing llms for automated extraction of detection rules from cloud-based cti," in *Proceedings of the ACM on Web Conference 2025 (WWW)*, 2025.
- [27] A. Alsaheel, Y. Nan, S. Ma, L. Yu, G. Walkup, Z. B. Celik, X. Zhang, and D. Xu, "ATLAS: A sequence-based learning approach for attack investigation," in *30th USENIX Security Symposium*, 2021.
- [28] B. Bhattarai and H. Huang, "SteinerLog: Prize collecting the audit logs for threat hunting on enterprise network," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security (AsiaCCS)*, 2022.
- [29] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in *2020 IEEE Symposium on Security and Privacy (S&P)*, 2020.
- [30] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, "Holmes: real-time apt detection through correlation of suspicious information flows," in *2019 IEEE Symposium on Security and Privacy (S&P)*, 2019.
- [31] S. T. King, Z. M. Mao, D. G. Lucchetti, and P. M. Chen, "Enriching intrusion alerts through multi-host causality," in *12th Annual Network and Distributed System Security Symposium (NDSS)*, 2005.
- [32] Y. Ji, S. Lee, M. Fazzini, J. Allen, E. Downing, T. Kim, A. Orso, and W. Lee, "Enabling refinable cross-host attack investigation with efficient data flow tagging and tracking," in *27th USENIX Security Symposium*, 2018.
- [33] N. F. Noy and C. D. Hafner, "The state of the art in ontology design: A survey and comparative review," *AI magazine*, vol. 18, 1997.
- [34] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *European Semantic Web Conference (ESWC)*, 2018.
- [35] R. Socher, D. Chen, C. D. Manning, and A. Ng, "Reasoning with neural tensor networks for knowledge base completion," *Advances in Neural Information Processing Systems (NIPS)*, vol. 26, 2013.
- [36] J. Caballero, G. Gomez, S. Matic, G. Sánchez, S. Sebastián, and A. Villacañas, "The rise of goodfat: A novel accuracy comparison methodology for indicator extraction tools," *Future Generation Computer Systems (FGCS)*, vol. 144, 2023.
- [37] Operationally transparent cyber (optc) data release. [Online]. Available: <https://github.com/FiveDirections/OpTC-data>